

Cómo ejecutar las pruebas automatizadas

Manual para principiantes — desde cero, sin asumir nada.

1

Instalas las herramientas

Node.js y los navegadores

2

Configuras el proyecto

dependencias y archivo .env

3

Ejecutas las pruebas

un solo comando

4

Revisas el reporte

verde = pasó, rojo = falló

¿Qué es esto y para qué sirve?

Este proyecto es un "robot" que abre un navegador, escribe, hace clic y verifica que la aplicación web funcione como se espera. En vez de que una persona repita la misma prueba a mano cada día, el robot la hace solo, en minutos, y deja evidencia (capturas, video y un reporte).

Tú no necesitas saber programar para EJECUTARLO. Solo necesitas seguir los pasos de este manual en orden, copiando y pegando los comandos tal cual aparecen.

Las 4 piezas que vas a instalar/usar

- Node.js — el motor que hace funcionar todo. Es como el "Windows" sobre el que corre el proyecto.
- npm — viene incluido con Node. Sirve para descargar piezas (librerías) y para lanzar los comandos del proyecto.
- Playwright — el que controla el navegador (Chrome) automáticamente.
- Cucumber — el que lee las pruebas escritas en lenguaje casi humano (Gherkin) y las ejecuta.



INFO — No te asustes con los nombres

Solo los mencionamos para que sepas qué hace cada cosa. Para ejecutar, basta con copiar los comandos. El glosario al final explica cada palabra.

¿Qué pasa cuando ejecutas una prueba?



0

Antes de empezar: lo que necesitas

Revisa que tengas esto. Si te falta algo, consíguelo antes de continuar:

- 1 Una computadora con Windows o Mac.
- 2 Conexión a internet (la primera instalación descarga varios archivos).
- 3 Alrededor de 2 GB de espacio libre en disco.
- 4 Permisos para instalar programas (si es del trabajo, quizás necesites pedirlo a IT).
- 5 La carpeta del proyecto en tu computadora (este mismo proyecto).



CONSEJO — Tómallo con calma

Haz un paso, verifica que funcionó, y recién entonces pasa al siguiente. Si algo falla, ve a la sección "Solución de problemas" al final.

1

Instalar Node.js

Node.js es el motor. Sin él, nada funciona. Se instala una sola vez.

- 1 Abre tu navegador y entra a: <https://nodejs.org>
- 2 Descarga el botón que dice "LTS" (es la versión estable y recomendada).
- 3 Abre el archivo descargado y haz clic en "Siguiente / Next" hasta el final. No cambies ninguna opción.
- 4 Cuando termine, cierra y vuelve a abrir cualquier terminal que tuvieras abierta.

Verifica que quedó instalado

Abre la terminal (te enseñamos cómo en el Paso 2) y escribe estos dos comandos. Si ves números de versión, ¡quedó bien!

```
Verificación de Node

PS> node --version
v20.11.1
PS> npm --version
10.2.4
```



ATENCIÓN — Si dice "no se reconoce el comando"

Node no quedó instalado o la terminal no se reinició. Cierra TODAS las terminales, ábrealas de nuevo y reintentá. Si persiste, reinstala Node.

2 Abrir la terminal

La "terminal" es una ventana donde escribes comandos (texto) y presionas Enter. Es la herramienta principal que vas a usar.

En Windows (PowerShell)

- 1 Presiona la tecla Windows.
- 2 Escribe: PowerShell
- 3 Haz clic en "Windows PowerShell" cuando aparezca.

En Mac (Terminal)

- 1 Presiona Command (Cmd) + barra espaciadora.
- 2 Escribe: Terminal
- 3 Presiona Enter.

INFO — Cómo se ve y cómo se usa

Verás una línea que termina en ">" o "\$". Ahí escribes el comando y presionas Enter. El símbolo "PS>" en este manual representa esa línea: NO lo escribas tú, solo lo que va después.

3 Entrar a la carpeta del proyecto

La terminal siempre está "parada" en alguna carpeta. Tienes que moverla hacia la carpeta del proyecto con el comando "cd" (change directory = cambiar de carpeta).



```
# Cambiar a la carpeta del proyecto (ajusta la ruta a la tuya)
PS> cd "C:\Users\tu-usuario\Desktop\Playwright_Web_Automation"
# Ver que estás en el lugar correcto (lista los archivos)
PS> dir
package.json  src  core  scripts  cucumber.js ...
```

CONSEJO — Truco para no escribir la ruta larga

Escribe "cd " (con espacio) y luego ARRASTRA la carpeta del proyecto desde el explorador hasta la terminal. La ruta se pega sola. Después presiona Enter.

LISTO — Cómo saber que estás bien parado

Si al escribir "dir" (Windows) o "ls" (Mac) ves el archivo package.json en la lista, estás en la carpeta correcta.

4

Instalar las dependencias del proyecto

El proyecto usa muchas piezas (librerías). Este comando las descarga todas dentro de una carpeta llamada `node_modules`. Se hace una sola vez (y de nuevo solo si cambian las librerías).

```
# Descarga TODAS las librerías que el proyecto necesita
PS> npm install
added 320 packages in 45s
found 0 vulnerabilities
```

Instalar dependencias



ATENCIÓN — Esto tarda unos minutos la primera vez

Es normal. Necesita internet. Si la conexión se corta, vuelve a ejecutar "npm install" y continúa donde quedó.

5

Instalar el navegador de Playwright

Playwright maneja un navegador "de prueba" separado del tuyo. Hay que descargarlo una vez con este comando:

```
# Descarga el navegador Chromium que usarán las pruebas
PS> npx playwright install chromium
Chromium 130.0 downloaded to ...
```

Instalar navegador



CUIDADO — Si más adelante ves "Executable doesn't exist"

Significa que faltó este paso. Vuelve aquí y ejecuta el comando de arriba.

6

Configurar el archivo de ambiente (.env)

El proyecto necesita saber CONTRA QUÉ aplicación correr (la URL) y, opcionalmente, datos de integraciones (Jira). Eso vive en un archivo llamado `.env.qa`. Hay una plantilla lista: `.env.example`. Solo cópiala y edítala.

```
# Crear tu archivo .env.qa a partir de la plantilla (Windows)
PS> copy .env.example .env.qa
# En Mac sería: cp .env.example .env.qa
```

Crear el .env.qa

Luego abre `.env.qa` con el Bloc de notas y, como mínimo, pon la URL de la aplicación. Para practicar puedes usar la demo pública de OrangeHRM:

```
BASE_URL=https://opensource-demo.orangehrmlive.com
ENV=qa
# Si NO usarás Jira, pon esta línea para que no intente conectarse:
JIRA_ENABLED=false
```

**CUIDADO — Nunca compartas ni subas a git los archivos .env**

Pueden contener contraseñas y tokens. Ya están excluidos del repositorio a propósito.

**INFO — ¿Qué pasa si falta el BASE_URL?**

Al ejecutar verás el error "BASE_URL is not defined". Significa que falta el archivo .env.qa o que esa línea está vacía.

7

Ejecutar las pruebas

¡Llegó el momento! Este es el comando principal. Abre el navegador, corre todos los escenarios y al final genera el reporte.

El comando principal

```
# Ejecuta TODAS las pruebas en el ambiente qa
PS> npm run test:qa
```

Vas a ver cómo el navegador se abre solo y "alguien invisible" escribe y hace clic. Al terminar, la terminal muestra un resumen como este:

Resumen al finalizar

```
11 escenarios (11 passed)
32 steps (32 passed)
0m48.231s
```

Otras formas útiles de ejecutar

Todas se escriben igual, cambiando el final:

- `npm run test:qa:video` -> igual, pero graba un video de cada escenario.
- `npm test -- --tags @Regresion` -> corre SOLO los escenarios marcados con @Regresion.
- `npm test -- src/test/features/login/Login.feature` -> corre SOLO ese archivo de pruebas.



CONSEJO — Acelerar o ir más lento

Por defecto corren 4 pruebas en paralelo. Para correr de una en una (más fácil de mirar): escribe `$env:PARALLEL=1` antes del comando, en la misma terminal (Windows PowerShell).

8

Revisar los resultados y la evidencia

Cuando termina, el proyecto crea varios materiales. Esto es lo que tienes y dónde encontrarlo:

Qué	Dónde	Para qué
Reporte HTML	reports/html/index.html	Se abre solo. Verde/rojo por escenario, con capturas.
Evidencia PDF	reports/pdf/	Un PDF por escenario, paso a paso.
Videos	test-results/videos/	Solo si grabaste video (o en CI / al fallar).
Traces	test-results/traces/	Diagnóstico técnico de los fallos.
JSON crudo	reports/cucumber-report.json	Lo usan el reporte y la sincronización con Jira.

El reporte HTML se abre solo en tu navegador. Ahí cada escenario aparece en VERDE si pasó o ROJO si falló, con las capturas paso a paso.



INFO — Para abrir un "trace" (diagnóstico de un fallo)

Ejecuta: `npx playwright show-trace test-results/traces/NOMBRE.zip` — se abre una vista interactiva con todo lo que pasó.



LISTO — ¿Cómo sé que todo salió bien?

El resumen de la terminal dice "(X passed)" y NO aparece ningún "(failed)". En el reporte HTML todo se ve verde.

Bonus: crear una prueba nueva (generador)

No tienes que escribir los archivos a mano. El proyecto incluye un generador que crea la estructura de una prueba nueva a partir de un "spec" (una receta en formato JSON) o respondiendo preguntas.

```
# Opción A: a partir de una receta ya escrita
PS> npm run new:module -- specs/mi-modulo.spec.json
# Opción B: el asistente te hace preguntas y arma todo
PS> npm run new:module
```

Generador de módulos

Detalles completos y la lista de campos están en docs/GENERATOR.md y en el ejemplo specs/example.spec.json.



CONSEJO — Flujo recomendado para el equipo

1) Copia specs/example.spec.json. 2) Cámbialo según tu caso de prueba. 3) Ejecuta el generador. 4) Abre el archivo Page que creó y completa los selectores donde diga // TODO. 5) Corre las pruebas.

Solución de problemas (errores comunes)

Si algo falla, busca aquí el mensaje parecido al que te salió:

Mensaje / Síntoma	Qué hacer
'node' no se reconoce	Node no está instalado o la terminal no se reinició. Reinstala Node y abre una terminal nueva.
npm install se queda pegado o falla	Revisa internet. Borra la carpeta node_modules y el archivo package-lock.json y reintentas "npm install".
BASE_URL is not defined	Falta el archivo .env.qa o la línea BASE_URL. Repite el Paso 6.
Executable doesn't exist	Falta el navegador. Ejecuta "npx playwright install chromium" (Paso 5).
TimeoutError / la prueba falla esperando	La página tardó mucho (común en la demo pública). Reintenta; si persiste, revisa tu internet o la URL.
Caracteres raros / acentos mal	Asegúrate de guardar .env y archivos en UTF-8.

Glosario (en palabras simples)

Término	Qué significa
Node.js	El motor que ejecuta el proyecto. Se instala una vez.
npm	Herramienta que viene con Node. Descarga librerías y lanza comandos ("npm run ...").
Dependencias	Las piezas/librerías que el proyecto necesita. Viven en node_modules.
Playwright	La herramienta que controla el navegador automáticamente.
Cucumber	Lee las pruebas escritas en lenguaje casi humano y las ejecuta.
Gherkin	El lenguaje de los archivos .feature: Given / When / Then (Dado / Cuando / Entonces).
Feature	Un archivo .feature con uno o varios escenarios de prueba.
Escenario (Scenario)	Un caso de prueba concreto: una secuencia de pasos.
Step	Un paso de un escenario (una línea Given/When/Then).
Page Object	Un archivo que representa una pantalla y sus acciones (login, etc.).
Locator	La "dirección" de un elemento en la pantalla (un botón, un campo).
Headless	Modo sin ver el navegador (corre invisible). Se usa en servidores/CI.
CI	Integración Continua: un servidor corre las pruebas solo en cada cambio.
.env	Archivo con la configuración (URL, tokens). No se comparte.
Trace	Grabación técnica de una prueba para diagnosticar por qué falló.



LISTO — ¡Eso es todo!

Con estos pasos cualquiera puede ejecutar las pruebas. Guarda este manual y compártelo con quien se sume al equipo.